BÖLÜM 6

AKIŞ TİPİ ATÖLYELERDE İŞ SIRALAMA YÖNTEMLERİ

İÇERİK

- ✓ **n İş-2 Serî Tezgâh (n×2) Durumu**
- ✓ **n İş-3 Serî Tezgâh (n×3) Durumu**
- ✓ **n İş-m Serî Tezgâh (n×m) Durumu**

n İş-2 Serî Tezgâh (n×2) Durumu
Johnson Yöntemi İle (n×2) Sıralama Problemlerinin Çözümü

- İki tezgâhtan geçecek n tane işin, aynı teknolojik sırayı izleyerek işlenmesi gerektiğini varsayalım. Bu tip serî iş akışı bulunan (n×2) sıralama problemlerinde bulunan en iyi iş sırasının şu ölçütleri eniyilemesi istenir:

  - Yayılma süresinin enküçüklenmesi
  - İşlerin ortalama bekleme süresinin enküçüklenmesi
  - Tezgâhların ortalama boş kalma süresinin enküçüklenmesi

Johnson Yöntemi İle (n×2) Sıralama Problemlerinin Çözümü

Bu istekleri gerçekleştiren, yâni optimal iş sırasının bulunması, Johnson yöntemi (J1-A) ile gerçekleştirilir. Bu yöntem şu adımlardan oluşur:

1. İlk tezgâhtaki işlem süreleri ikinci tezgâhtaki işlem sürelerine göre küçük olan işler küçükten büyüğe sıralanarak bir alt sıra elde edilir. İlgili iş her iki tezgâhta da aynı süreye sahipse bu adıma veya ikinci adıma dahil edilebilir.
2. İkinci tezgâhtaki işlem süreleri birinci tezgâhtaki işlem sürelerine göre küçük olan işler büyükten küçüğe sıralanarak ikinci alt sıra elde edilir.
3. Bu iki alt sıra, ardışık olarak birleştirilir.

Johnson Yöntemi İle (n×2) Sıralama Problemlerinin Çözümü

Johnson yönteminin bir başka uygulama şekli (J1-B) ise şu adımlardan oluşmaktadır:

1. Tüm işlem süresi tablosundaki en küçük süre değeri bulunur. Eğer eşit en küçük iki süre değeri varsa herhangi biri seçilebilir.
2. Bu süre birinci tezgâha ilişkin ise, bu sürenin sırasındaki iş, ilk iş olarak belirlenir. İkinci tezgâha ilişkin ise, bu sürenin sırasındaki iş sonuncu iş olarak belirlenir.
3. Bu işin sırası çizilerek tablodan çıkarılır.
4. 1-3 arası adımlar, tüm işler sıralanıncaya dek yinelenir.

Örnek 6.1.

5 iş (i), 2 tezgâhta (T_1-T_2) tezgâh sırasıyla yapılmaktadır. Bu tezgâhlardaki işlem süreleri (ZB), Tablo 6.1.'de verilmiştir. Yayılma süresini enküçükleyen iş sırasını belirleyiniz.

Tablo 6.1. Örnek 6.1. ile ilgili tablo

i	T_1	T_2
1	4	3
2	1	2
3	5	4
4	2	3
5	5	6

Örnek 6.1.

i	T_1	T_2
1	4	3
2	1	2
3	5	4
4	2	3
5	5	6

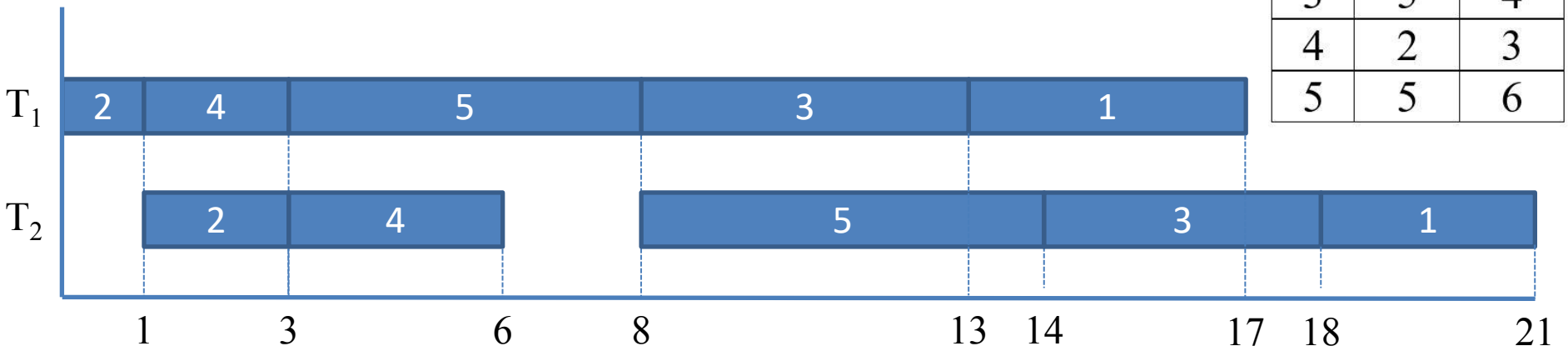
i	T_1	T_2
1	4	3
2	1	2
3	5	4
4	2	3
5	5	6

J1-A yöntemini uygulayalım:

1. 2, 4 ve 5 no.'lu işlerin birinci tezgâhtaki (T_1) süreleri, ikinci tezgâhtaki (T_2) sürelerinden küçüktür. Bu işler, ($P_{21} \leq P_{41} \leq P_{51}$) olduğundan (2-4-5) şeklinde sıralanır.
2. 1 ve 3 no.'lu işlerin ikinci tezgâhtaki süreleri, birinci tezgâhtaki sürelerinden küçüktür. Bu işler, ($P_{32} \geq P_{12}$) olduğundan (3-1) şeklinde sıralanır.
3. Bu işleri en kısa zamanda bitirecek iş sırası, bulunan bu iki sırayı bozmadan birleştirmek sûretiyle bulunur: (2-4-5-3-1)

Örnek 6.1.

i	T_1	T_2
1	4	3
2	1	2
3	5	4
4	2	3
5	5	6



Şekil 6.1. Örnek 6.1. ile ilgili Gantt sıralama diyagramı (2-4-5-3-1 iş sırası)

Yayılma Süresi: 21 ZB

T_1 hiç boş kalmıyor.

T_2 ise 2 no.'lu iş için 1, 5 no.'lu iş için ise 2 olmak üzere toplam 3 ZB boş kalmakta.

Örnek 6.2.

5 iş (A, B, C, D, E), 2 tezgâhta (T_1-T_2) tezgâh sırasıyla yapılmaktadır. Bu tezgâhlardaki işlem süreleri (ZB), hazırlık süreleri dahil olmak üzere Tablo 6.2.'de verilmiştir. Yayılma süresini enküçükleyen iş sırasını belirleyiniz.

Tablo 6.2. Örnek 6.2. ile ilgili tablo

	İşlem Süreleri (ZB)	
İş	**T_1**	**T_2**
A	3	2
B	6	3
C	9	8
D	4	6
E	7	4

Örnek 6.2.

J1-B yöntemini uygulayalım:

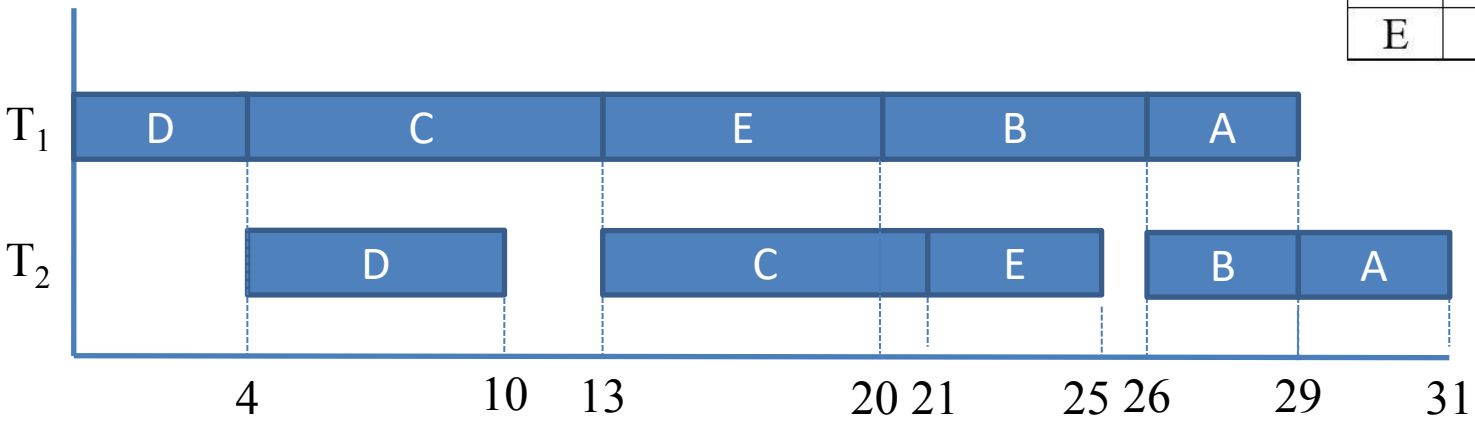
Tablo 6.2. Örnek 6.2. ile ilgili tablo

	İşlem Süreleri (ZB)		
İş	**T_1**	**T_2**	**Sıra**
A	3	2	5
B	6	3	4
C	9	8	2
D	4	6	1
E	7	4	3

En İyi Sıra: D-C-E-B-A

Örnek 6.2.

	İşlem Süreleri (ZB)	
İş	T_1	T_2
A	3	2
B	6	3
C	9	8
D	4	6
E	7	4



Şekil 6.2. Örnek 4.2. ile ilgili Gantt sıralama diyagramı (D-C-E-B-A iş sırası)

Yayılma Süresi: 31 ZB

T_1 hiç boş kalmıyor.

T_2 ise D işi için 4, C işi için 3 ve B işi için ise 1 olmak üzere toplam 8 ZB boş kalmakta.

Johnson Yönteminin Ön/Art Geri Kalma Durumları İçin Geliştirilmesi

Klasik (n×2) sıralama probleminde;

- beklemeler,
- gecikmeler,
- zaman kaymaları

gibi gerçek yaşamdaki aksaklıklar dikkate alınmamaktadır.

Johnson Yönteminin Ön/Art Geri Kalma Durumları İçin Geliştirilmesi

"Geliştirme" ifâdesi ile kastedilen durumda ise problem şu durumları da içerir hâle getirilir:

- Bekleme sürelerinin modele dahil edilmesi
- Teslim Gecikmesi (tardiness) ve Erken Tamamlama (earliness) cezalarının eklenmesi
- Daha fazla makina veya esnek üretim ortamlarına uyarlanması
- Gerçek üretim koşullarına (stok, ara bekleme, taşıma süreleri) uygun duruma getirilmesi

Johnson Yönteminin Ön/Art Geri Kalma Durumları İçin Geliştirilmesi

- İş sıralama problemleri için kullanılabilecek varsayımlardan biri, bir tezgâhta başlatılan bir işlemin, bitinceye dek aralıksız sürdürülmesi (bölünemez iş durumu) olabilir. Partiler hâlinde üretim veya tek tek ürün üretiminde, aşağıdaki nedenlerle bu varsayımı kaldırmak olanaklıdır:

1. Partiler hâlindeki üretimde parti tümüyle işlenmeden, işlenmiş olan bir kısmı, bir sonraki işleme geçebilir. Örneğin bir inşaat işinde inşaat sürerken, diğer yandan doğrama ve boyama işlemine başlanabilir.

2. Tek tek ürün üretiminde ise sıra bir işlemde iken, bir sonraki işleme ilişkin hazırlıklar bu anda yapılarak zaman kazanılmış olur (Örneğin takım, donanım veya tezgâhın hazırlanması).

Johnson Yönteminin Ön/Art Geri Kalma Durumları İçin Geliştirilmesi

- Bir işe ilişkin bir işlem yapılırken bir sonraki işleme geçilebilmesi için, ilk işin başlamasından itibâren belirli bir sürenin geçmesi gerekir. Bu süreye başlama geciktirmesi (ön geri kalma süresi) denir.

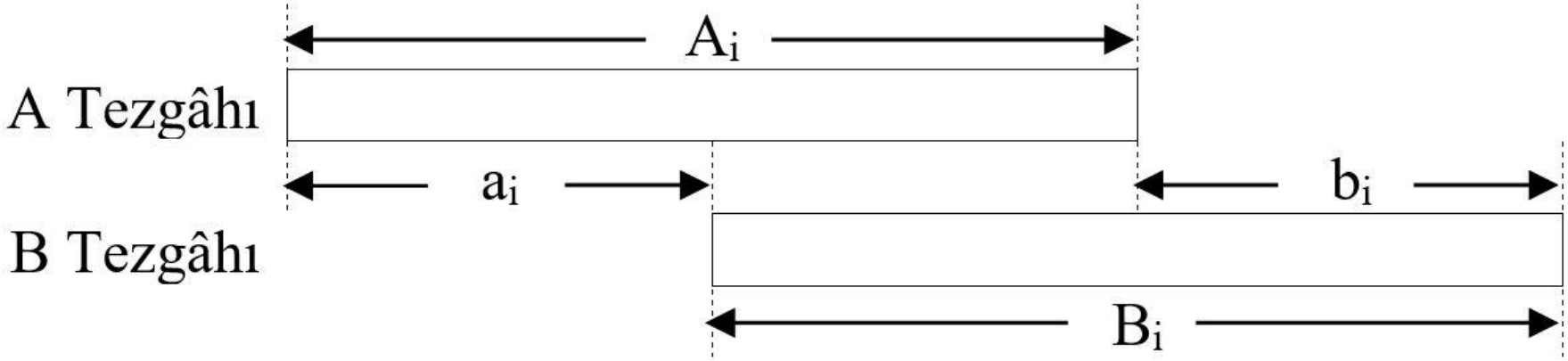
- Aynı şekilde bir sonraki işlemin bitirilebilmesi için, bir önceki işlemin bitişinden itibâren belirli bir süre geçmesi gerekir. Bu süreye ise bitiş geciktirmesi (art geri kalma süresi) denir.

Johnson Yönteminin Ön/Art Geri Kalma Durumları İçin Geliştirilmesi

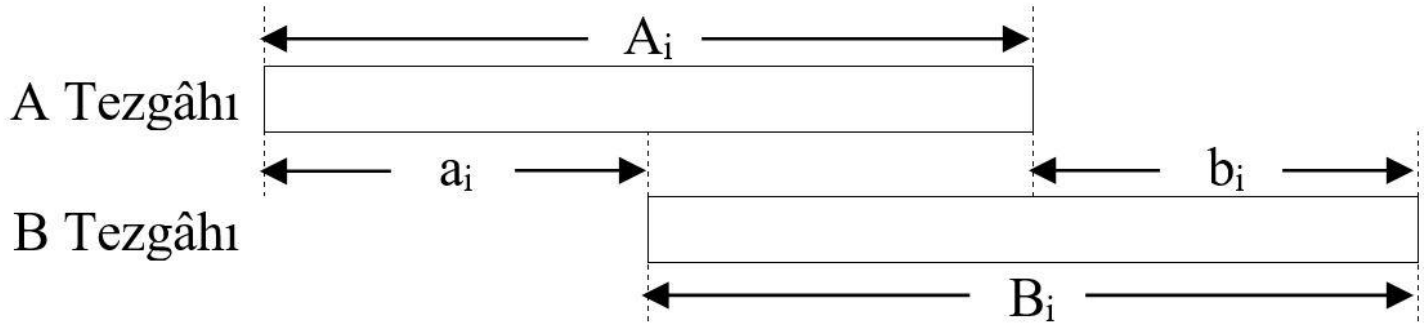
- Aynı teknolojik sırayı izleyen ve iki tezgâhta işlenen (örneğin A-B tezgâh sırası) n tane işin her biri için bu süreleri şekil üzerinde göstermeye çalışalım.

Johnson Yönteminin Ön/Art Geri Kalma Durumları İçin Geliştirilmesi

A_i : i işinin A tezgâhındaki işlem süresi
a_i : i işi için başlama geciktirmesi
B_i : i işinin B tezgâhındaki işlem süresi
b_i : i işi için bitiş geciktirmesi



Şekil 6.3. İşlem süreleri ile ön ve art geri kalma sürelerinin ilişkisi



Burada a_i ve b_i değerleri, geciktirme sürelerinin en küçük değerleridir. $(A_i–a_i)$ ile $(B_i–b_i)$ değerlerinin aynı olması gerekmez.

Örneğin (A=4) ve (a=1) ise bunun anlamı şudur: B'nin başlaması için A'nın başlamasından sonra en az 1 ZB'lik süre geçmesi gerekir. Ancak B, bundan daha fazla bir süre sonra başlayabilir, yâni daha fazla geciktirilebilir.

B'nin en geç başlama zamanı da, bitiş geciktirmesi ile belirlenir. Eğer (B=5) ve (b=3) ise, bu durumda B, A'nın bitiminden en az 3 ZB geçtikten sonra bitebilir. Diğer bir deyişle, A'nın başlamasından (5–3=) 2 ZB sonra B başlayabilir.

Johnson Yönteminin Ön/Art Geri Kalma Durumları İçin Geliştirilmesi

Bu tip geciktirme süreleri belli ise (n×2) iş sıralama problemlerinin çözümü için aşağıdaki adımlar uygulanır:

1. Tüm A_i, a_i, B_i ve b_i değerleri listelenir.
2. Tüm işler için $(a_i–A_i)$ ve $(b_i–B_i)$ farkları hesaplanır.
3. g_i = Enb $[(a_i–A_i); (b_i–B_i)]$ değerleri hesaplanır.
4. A_i değeri B_i değerinden küçük olan M iş kümesi belirlenir.
5. M kümesi için (A_i+g_i) değerleri hesaplanır.
6. Adım-5'de bulunan değerlere göre işler, artan büyüklükte sıraya dizilir [1, 2, …, k].

►

Johnson Yönteminin Ön/Art Geri Kalma Durumları İçin Geliştirilmesi

7. B_i değeri A_i değerinden küçük olan N iş kümesi belirlenir.
8. N kümesi için (B_i+g_i) değerleri hesaplanır.
9. Adım-8'de bulunan değerlere göre işler, azalan büyüklükte sıraya dizilir [k+1, k+2, …, n].
10. Aranan iş sırası [1, 2, …, k, k+1, k+2, …, n] olup bu, optimum iş sırasıdır.

NOT: A_i değeri B_i değeri ile eşitse herhangi biri seçilebilir, M veya N kümelerinden herhangi birine konur.

Örnek 6.3.

Sırasıyla A ve B tezgâhlarında işlenecek olan 5 işe (1, 2, 3, 4, 5 no.'lu işlere) ilişkin işlem süreleri ile ön ve art geri kalma süreleri, Tablo 6.3.'de verilmiştir. Yayılma süresini enküçükleyen iş sırasını belirleyiniz.

Tablo 6.3. Örnek 6.3. ile ilgili tablo (1)

i	A_i	a_i	B_i	b_i
1	16	2	8	1
2	8	4	10	2
3	5	3	9	2
4	4	5	12	4
5	8	6	6	4

Örnek 6.3.

i	A_i	a_i	B_i	b_i
1	16	2	8	1
2	8	4	10	2
3	5	3	9	2
4	4	5	12	4
5	8	6	6	4

Tablo 6.4. Örnek 6.3. ile ilgili tablo (2)

i	a_i-A_i	b_i-B_i	g_i	$M(A_i+g_i)$	Sıra	$N(B_i+g_i)$	Sıra	Genel İş Sırası
1	-14	-7	-7			1	5	5
2	-4	-8	-4	4	2			2
3	-2	-7	-2	3	1			1
4	1	-8	1	5	3			3
5	-2	-2	-2			4	4	4

Görüldüğü gibi optimum iş sırası (3-2-4-5-1) sırasıdır.

Örnek 6.4.

Bir bilezik üreticisinin, önemli bir müşterisi için sıraya sokması gereken 5 sipârişi vardır. Her sipârişteki işler, kalıplama ve onu izleyen son şekillendirme adımlarından geçerek yapılmaktadır.

Bir sipâriş partisindeki herhangi bir parça üzerinde yapılan kalıplama işlemi tamamlandıktan hemen sonra, o partinin son şekillendirme işlemi başlayabilmektedir.

Örnek 6.4.

Tablo 6.5.'de;

- sipâriş partilerindeki her iş için parça başına işlem süreleri (dk)
- kalıplama aşaması öncesinde yapılması gereken işlemler için hazırlık süreleri (dk)
- sipâriş parti büyüklükleri (adet)

verilmiştir.

5 sipârişi, yayılma süresini enküçükleyecek şekilde sıralayınız.

Örnek 6.4.

Tablo 6.5. Örnek 6.4. ile ilgili tablo (1)

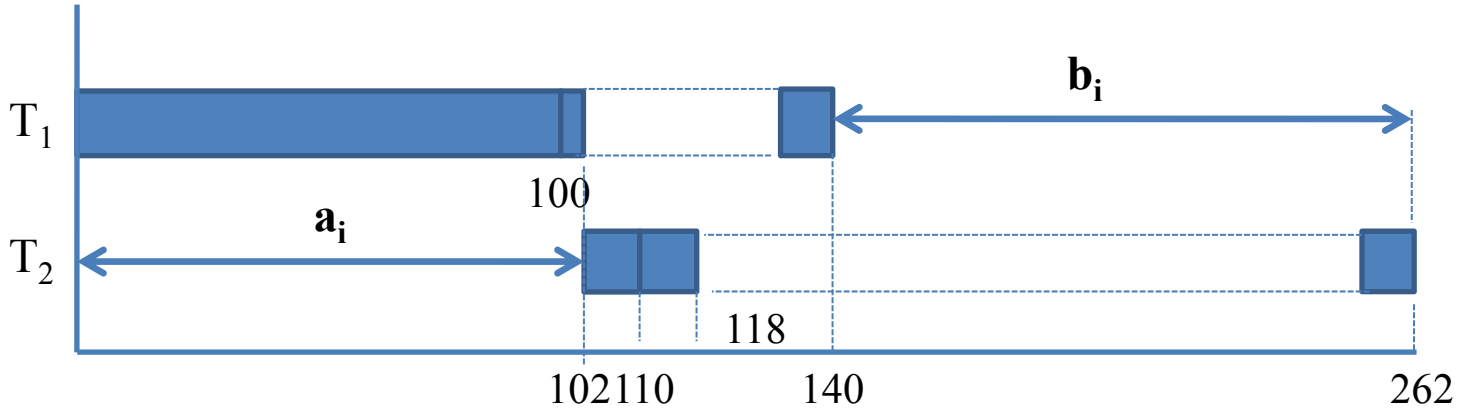
		İşlem Süreleri		
Sipâriş	**Parti Büyüklüğü**	**Kalıplama**	**Son Şekillendirme**	**Hazırlık Süresi**
1	20	2	8	100
2	25	2	5	250
3	100	1	2	60
4	50	4	2,5	60
5	40	3	6	80

Örnek 6.4.

Sipâriş	Parti Büyüklüğü	İşlem Süreleri		Hazırlık Süresi
		Kalıplama	Son Şekillendirme	
1	20	2	8	100
2	25	2	5	250
3	100	1	2	60
4	50	4	2,5	60
5	40	3	6	80

Kalıplama: A, Son Şekillendirme: B (Tüm sipârişler için)

1. sipârişin **A işlemi 100. dk'da başlayacak** ve **1. parçanın A işlemi**, (100+2=) **102. dk sonunda bitecektir**. Bu sipârişe ilişkin B işlemi ise bu anda başlayacaktır. **A işlemi** (100+20×2=) **140. dk'da**, B işlemi ise (102+20×8=) 262. dk'da sona erecektir.



A_i =140 a_i =102 B_i = (262-102=) 160 b_i = (262-140=) 122

Şekil 6.4. Örnek 6.4.'de verilen sipâriş-1 için Gantt diyagramı

Örnek 6.4.

Sipâriş	Parti Büyüklüğü	İşlem Süreleri		Hazırlık Süresi
		Kalıplama	Son Şekillendirme	
1	20	2	8	100
2	25	2	5	250
3	100	1	2	60
4	50	4	2,5	60
5	40	3	6	80

Sipâriş-2:

$A_i = 250 + 25\times2 = 300$

$a_i = 250 + 2 = 252$

$B_i = 25\times5 = 125$

$b_i = 252 + 125 - 300 = 77$

Sipâriş-3:

$A_i = 60 + 100\times1 = 160$

$a_i = 60 + 1 = 61$

$B_i = 100\times2 = 200$

$b_i = 61 + 200 - 160 = 101$

Sipâriş-5:

$A_i = 80 + 40\times3 = 200$

$a_i = 80 + 3 = 83$

$B_i = 40\times6 = 240$

$b_i = 83 + 240 - 200 = 123$

Örnek 6.4.

Sipâriş	Parti Büyüklüğü	İşlem Süreleri		Hazırlık Süresi
		Kalıplama	Son Şekillendirme	
1	20	2	8	100
2	25	2	5	250
3	100	1	2	60
4	50	4	2,5	60
5	40	3	6	80

Sipâriş-4:

Bu sipârişteki her iş için A işlemi, B işleminden daha uzun olduğu için, ikinci tezgâhta sürekli bekleme oluşacaktır.

$A_i = 60 + 50 \times 4 = 260$

$a_i = 60 + 4 = 64$

$B_i = 260 + 2,5 - 64 = 198,5$

$b_i = 262,5 - 260 = 2,5$

Örnek 6.4.

Sipâriş	Parti Büyüklüğü	İşlem Süreleri		Hazırlık Süresi
		Kalıplama	Son Şekillendirme	
1	20	2	8	100
2	25	2	5	250
3	100	1	2	60
4	50	4	2,5	60
5	40	3	6	80

Tablo 6.6. Örnek 6.4. ile ilgili tablo (2)

i	a_i	A_i	b_i	B_i	$a_i - A_i$	$b_i - B_i$	g_i	$M(A_i + g_i)$	Sıra	$N(B_i + g_i)$	Sıra	Genel Sıra
1	102	140	122	160	-38	-38	-38	102	3			3
2	252	300	77	125	-48	-48	-48			77	4	4
3	61	160	101	200	-99	-99	-99	61	1			1
4	64	260	2,5	198,5	-196	-196	-196			2,5	5	5
5	83	200	123	240	-117	-117	-117	83	2			2

En iyi sipâriş sırası: 3-5-1-2-4

n İş-3 Serî Tezgâh (n×3) Durumu
Johnson Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

- n işin her biri, sırasıyla A-B-C tezgâhlarından geçiyorsa; yayılma süresini enküçükleyen iş sırasını saptamak üzere geliştirilmiş etkin ve pratik bir teknik yoktur.

- Ancak bâzı özel durumlar için problemin en iyi çözümünü bulmak olanaklıdır. Belirli koşullar sağlandığı takdirde J1-A veya J1-B'nin geliştirilmiş şekli (J2) ile işlerin en kısa süre içinde bitirilmesini sağlayacak optimal iş sırasının bulunması kolaydır.

- Bu koşullar sağlanmadığı takdirde, analitik yöntemlerle işlerin en kısa zamanda bitirilmesini sağlayacak iş sırasının bulunması zor ve zaman alıcı olabilir.

Johnson Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

J2 yönteminin uygulanabilmesi için

- P_{iA} : i. işin A tezgâhındaki işlem süresi
- P_{iB} : i. işin B tezgâhındaki işlem süresi
- P_{iC} : i. işin C tezgâhındaki işlem süresi

olmak üzere aşağıdaki iki koşuldan birinin sağlanması gerekir:

- Enk $P_{iA} \geq$ Enb P_{iB}
- Enk $P_{iC} \geq$ Enb P_{iB}

Johnson Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

Kısaca J2 yönteminin uygulanabilmesi için 2. tezgâhın (B) tümüyle 1. tezgâh (A) veya 3. tezgâh (C) ile süre olarak birleştirilmesi gerekir. İki koşuldan biri sağlandığı zaman iki yapay tezgâh (A^* ve C^*) tanımlanır. Bu tezgâhlardaki işlem süreleri ise şu şekilde bulunur:

$$P_{iA}^* = P_{iA} + P_{iB}$$

$$P_{iC}^* = P_{iB} + P_{iC}$$

Bu tanımlamadan sonra bu iki yapay tezgâh kullanılarak J1-A veya J1-B yöntemiyle sıralama yapılır.

Örnek 6.5.

5 parça (i), sırasıyla A-B-C tezgâhlarında işlenerek üretilmektedir. Bu üretim ile ilgili işlem (hazırlık+işlenme) süreleri (P_{iA}, P_{iB}, P_{iC}), Tablo 6.7.'de verilmiştir.

Yayılma süresini enküçükleyen iş sırasını belirleyiniz.

Tablo 6.7. Örnek 6.5. ile ilgili tablo (1)

i	P_{iA}	P_{iB}	P_{iC}
1	5	4	4
2	7	2	6
3	3	1	7
4	2	1	10
5	4	4	8

Örnek 6.5.

i	P_{iA}	P_{iB}	P_{iC}
1	5	4	4
2	7	2	6
3	3	1	7
4	2	1	10
5	4	4	8

J2 yöntemini kullanalım:

Enk $P_{iA} = 3$

Enb $P_{iB} = 4$

Enk $P_{iC} = 4$

Tablo 6.8.

i	$P_{iA} + P_{iB}$	$P_{iB} + P_{iC}$	Sıra
1	9	8	5
2	9	8	4
3	4	8	2
4	3	11	1
5	8	12	3

olup (Enk $P_{iC} \geq$ Enb P_{iB}) koşulu sağlanmaktadır. A^* ve C^* yapay tezgâhlarını Tablo 6.8.'deki gibi oluşturabiliriz.

En iyi sıra (4-3-5-2-1) da aynı tabloda J1-A yöntemiyle belirlenmiştir.

Johnson Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

- Giglio ve Wagner, J2 yöntemini, bu yöntemin uygulanması için gerekli koşullardan hiç birini sağlamayan 20 adet (6×3) tipi serî akışlı iş sıralama problemine uygulamışlardır.

- Bu problemlerin 9'unda J2 yöntemi en iyi çözümü vermiştir. Geri kalan problemlerin 8'nde ise elde edilen sıralamadaki 2 işin yer değiştirmesiyle en iyi çözüm elde edilmiştir.

- Kalan 3 problemde ise elde edilen çözüm aynı yayılma süresini veren en iyi birkaç çözümden biridir.

Johnson Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

- (n×3) sıralama problemlerinde J2 yöntemi, en iyi çözüm koşulları sağlanmasa da en iyi ya da en iyiye yakın çözümler vermektedir. En azından elde edilen çözüm, analitik yöntemlerin kullanılabilmesi için iyi bir başlama noktası oluşturmaktadır.

Johnson Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

- J2 yönteminin özel bir çözümü vardır.

- İlk iki tezgâha (A ve B) ve son iki tezgâha (B ve C), koşullar dikkate alınmaksızın J1-A veya J1-B yöntemi uygulandığında, her iki sonuç da aynı sırayı veriyorsa bu iş sırası optimaldir.

Örnek 6.6.

Tablo 6.9.'da (5×3) tipi bir sıralama problemi verileri görülmektedir.

Tablo 6.9. Örnek 6.6. ile ilgili tablo (1)

i	P_{iA}	P_{iB}	P_{iC}
1	26	20	19
2	7	18	42
3	34	4	1
4	27	44	30
5	21	22	32

Örnek 6.6.

i	P_{iA}	P_{iB}	P_{iC}
1	26	+20	*19
2	7+	18*	42
3	34	+4	*1
4	27+	44	*30
5	21+	22*	32

(A-B) tezgâh ikilisine J1-A yöntemi uygulandığında (**2-5-4-1-3**) iş sırası elde edilir.

Aynı şekilde (B-C) tezgâh ikilisine de J1-A yöntemi uygulandığında (**2-5-4-1-3**) iş sırası elde edilir.

Dolayısıyla bu sıra, optimal iş sırasıdır.

Szwarc Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

- J2 yöntemi, Wlodzimierz Szwarc tarafından, farklı özel koşullar dikkate alınarak geliştirilmiştir.
- Bu bölümde, bu koşullar ve problemin koşullara uyması durumunda çözüm için gerekli adımlar anlatılacaktır.
- i (i=1,2,…,n) işinin üç tezgâhtan (A-B-C) tezgâh sırasıyla geçmesi problemini yeniden ele alalım.
- A_i, B_i ve C_i ise, bu tezgâhlarda i işinin işlem süreleri olsun.

Szwarc Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

Koşul-1 : $\text{Enk } B_i \geq \text{Enk } [\text{Enb } A_i; \text{Enb } C_i]$

Bu eşitsizliğin sağ tarafını iki ayrı durum olarak inceleyelim:

Koşul-1a: $\text{Enk } B_i \geq \text{Enb } A_i$

Bu koşul sağlanırsa aşağıdaki adımlar uygulanır:

1. B ve C tezgâhlarında J1-A veya J1-B yöntemi uygulanarak bir iş sırası belirlenir.
2. 1. adımda bulunan sıradaki ilk işin A tezgâhındaki işlem süresi ($A_{[1]}$) ile diğer işlerin A tezgâhındaki işlem süreleri (A_i) karşılaştırılır. ($A_i<A_{[1]}$) koşulunu sağlayan her i işi, bulunduğu konumdan ilk sıraya getirilerek yeni iş sıraları elde edilir.
3. 1. ve 2. adımlarda elde edilen tüm iş sıraları için, yayılma süresi en az olan saptanır. Bu sıra, optimal iş sırasıdır.

Örnek 6.7.

Tablo 6.10.'da (6×3) tipi bir sıralama problemi verileri görülmektedir.

Tablo 6.10. Örnek 6.7. ile ilgili tablo

i	A_i	B_i	C_i
1	4	6	2
2	5	7	7
3	3	5	10
4	5	8	3
5	2	9	6
6	1	6	10

Örnek 6.7.

i	A_i	B_i	C_i
1	4	6	+2
2	5	7+	7
3	3	5+	10
4	5	8	+3
5	2	9	+6
6	1	6+	10

Enk B_i (=5) ≥ Enb A_i (=5)

Adım-1 : (3-6-2-5-4-1)

Adım-2 : (5-3-6-2-4-1) ve (6-3-2-5-4-1)

Bu sıralar için yayılma süresini, Gantt diyagramlarını kullanarak belirlersek şu değerleri elde ederiz:

(3-6-2-5-4-1) : 46

(5-3-6-2-4-1) : 49

(6-3-2-5-4-1) : 45

Optimal sıra, yayılma süresi daha küçük olan (6-3-2-5-4-1) iş sırasıdır.

Szwarc Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

Koşul-1 : **Koşul-1b:** **Enk $B_i \geq$ Enb C_i**

Bu koşul sağlanırsa aşağıdaki adımlar uygulanır:

1. A ve B tezgâhlarında J1-A veya J1-B yöntemi uygulanarak bir iş sırası belirlenir.
2. Adım 1'de bulunan sıradaki son işin C tezgâhındaki işlem süresi ($C_{[n]}$) ile diğer işlerin C tezgâhındaki işlem süreleri (C_i) karşılaştırılır. ($C_i<C_{[n]}$) koşulunu sağlayan her i işi, bulunduğu konumdan son sıraya getirilerek yeni iş sıraları elde edilir.
3. Adım-1 ve adım-2'de elde edilen tüm iş sıraları için, yayılma süresi en az olan saptanır. Bu sıra, optimal iş sırasıdır.

Örnek 6.8.

Tablo 6.11.'de verilen (5×3) tipi iş sıralama problemi, Koşul-1b'yi sağlayan işlem sürelerine sahiptir. **Optimal iş sırası ?**

Tablo 6.11. Örnek 6.8. ile ilgili tablo

i	A_i	B_i	C_i
1	2	8	3
2	9	9	7
3	12	7	2
4	3	11	4
5	10	8	3

Örnek 6.8.

i	A_i	B_i	C_i
1	2*	8	3
2	9*	9	7
3	12	*7	2
4	3*	11	4
5	10	*8	3

Enk B_i (=7) ≥ Enb C_i (=7)

Adım-1 : (1-4-2-5-3)

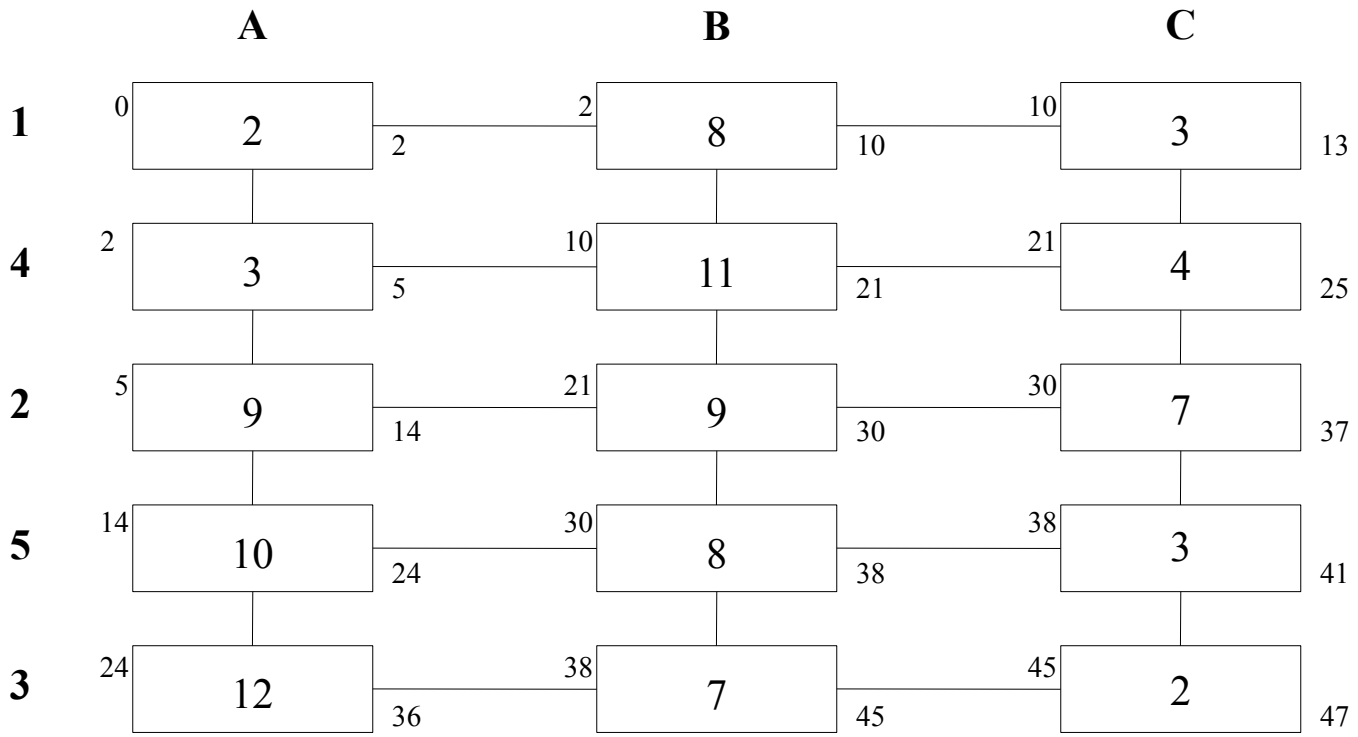
Adım-2 : –

Tek sıra elde edildi.

(1-4-2-5-3) sırası için yayılma süresi 47 olup bu sıra, optimal iş sırasıdır.

Örnek 6.8.

i	A_i	B_i	C_i
1	2	8	3
2	9	9	7
3	12	7	2
4	3	11	4
5	10	8	3



Szwarc Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

Koşul-2 : $\text{Enk } B_i \geq \text{Enb } (\text{Enb } A_i; \text{Enb } C_i)$

Bu koşula dikkat edildiğinde;

$$\text{Enk } B_i \geq \text{Enb } A_i$$

$$\text{Enk } B_i \geq \text{Enb } C_i$$

koşullarının her ikisini birden içerdiği anlaşılmaktadır.

Bu koşullar sağlanırsa aşağıdaki işlem yapılır:

($i \neq j$) olmak üzere tüm i ve j'ler için;

$$\text{Enk } (A_i + C_j) = A_r + C_s \qquad (6.1)$$

koşulunu sağlayan r ve s işleri bulunur.

(S = r, …, s) şeklindeki her iş sırası optimaldir.

Örnek 6.9.

Tablo 6.12.'de verilen (4×3) tipi iş sıralama problemi, Koşul-2'yi sağlayan işlem sürelerine sahiptir. **Optimal iş sırası ?**

Tablo 6.12. Örnek 6.9. ile ilgili tablo

i	A_i	B_i	C_i
1	5	10	7
2	8	12	6
3	7	9	2
4	4	14	9

Örnek 6.9.

i	A_i	B_i	C_i
1	5	10	7
2	8	12	6
3	7	9	2
4	4	14	9

Enk B_i (=9) ≥ [Enb (Enb A_i; Enb C_i) = Enb (8; 9)] (= 9)

Adım-1 : Enk $(A_i+C_j) = A_4 + C_3 = 4 + 2 = 6$

$i \neq j$

(r=4, s=3 olarak bulunur.)

Adım-2 : Optimal sıralar şunlardır: (4-1-2-3) ve (4-2-1-3)

Szwarc Yöntemi İle (n×3) Sıralama Problemlerinin Çözümü

Koşul-3 : $\mathbf{B_i = B_{i+1},\ A_i \leq A_{i+1},\ C_i \geq C_{i+1}}$

Bu koşulu aşağıdaki gibi açabiliriz:

$$\mathbf{B_1 = B_2 = \ldots = B_n}$$

$$\mathbf{A_1 \leq A_2 \leq \ldots \leq A_n}$$

$$\mathbf{C_1 \geq C_2 \geq \ldots \geq C_n}$$

Bu koşullar sağlanırsa (S=1,2,…,n) iş sırası optimaldir.

Örnek 6.10.

Tablo 6.13.'de verilen (5×3) tipi iş sıralama problemi, Koşul-3'ü sağlayan işlem sürelerine sahiptir. **Optimal iş sırası ?**

Tablo 6.13. Örnek 6.10. ile ilgili tablo

i	A_i	B_i	C_i
1	5	6	14
2	8	6	10
3	11	6	5
4	3	6	16
5	9	6	7

Örnek 6.10.

i	A_i	B_i	C_i
1	5	6	14
2	8	6	10
3	11	6	5
4	3	6	16
5	9	6	7

$$B_1 = B_2 = B_3 = B_4 = B_5 \ (=6)$$

$$A_4 \ (=3) \leq A_1 \ (=5) \leq A_2 \ (=8) \leq A_5 \ (=9) \leq A_3 \ (=11)$$

$$C_4 \ (=16) \geq C_1 \ (=14) \geq C_2 \ (=10) \geq C_5 \ (=7) \geq C_3 \ (=5)$$

Optimal sıra, (4-1-2-5-3) iş sırasıdır.

n İş-m Serî Tezgâh (n×m) Durumu

CDS Algoritması İle (n×m) Sıralama Problemlerinin Çözümü

n işin ve m tezgâhın (n>1, m>3) bulunduğu statik ve akış tipi sıralama problemlerinde, yayılma süresini mâkul sürede enküçükleyecek iş sırasının belirlenmesi için kesin bir yöntem yoktur (Dal-Sınır Algoritması vb. gibi yöntemler optimal çözümü uzun sürede bulurlar).

(n!) olası çözümden en az birinin, verilen performans ölçütünü eniyilemesi, hesaplama açısından büyük zorluklar getirir.

Bu problemlerin çözümü için en iyi çözüme yakınsayan bâzı yöntemler geliştirilmiştir.

CDS Algoritması ile (n×m) Sıralama Problemlerinin Çözümü

Campbell, Dudek ve Smith'in geliştirdiği bir yöntem (CDS algoritması), bu tip problemlerde başarı sağlamıştır.

Bu yöntem, (n×3) sıralama problemlerinde olduğu gibi iş sürelerinde toplamlar elde ederek Johnson yönteminin (J3) kullanımını sağlamaktadır.

Bu yaklaşımda, m tezgâhlı bir sistemde (m–1) adet grupta 2 kümeli iş süreleri elde edilmekte ve bu kümelerin her birine J1-A veya J1-B yöntemi uygulanmaktadır. Ortaya çıkan olası tüm sonuçlar dikkate alınmalıdır.

Bu çözümlemede n tane işin tezgâhlardan aynı sırada geçtikleri varsayılmaktadır.

Çözümlemenin genel yapısı;

t_m : İşlerin m no.'lu tezgâhtaki işlem sürelerinin kümesi

olmak üzere tablodaki gibidir.

Grup	Küme 1	Küme 2
1	t_1	t_m
2	$t_1 + t_2$	$t_m + t_{(m-1)}$
. . .		
k	$t_1 + t_2 + \ldots + t_k$	$t_m + t_{(m-1)} + \ldots + t_{(m+1-k)}$
. . .		
m–1	$t_1 + t_2 + \ldots + t_{(m-1)}$	$t_m + t_{(m-1)} + \ldots + t_2$

CDS Algoritması ile (n×m) Sıralama Problemlerinin Çözümü

Nihaî sonuca ulaşabilmek için ardışık olarak şu iş demetlerini yapmak gereklidir:

- Birbirinden farklı olası tüm iş sıralarını belirlemek.
- Bu sıraların her biri için yayılma süresini hesaplamak.
- En kısa yayılma süresini veren iş sırasını/sıralarını seçmek.

Örnek 6.11.

6 parça (a, b, c, d, e, f), dört tezgâhtan (T_1-T_2-T_3-T_4) tezgâh sırasıyla geçerek işlenmekte olup işlem süreleri (ZB), Tablo 6.15.'de verilmiştir.

Tablo 6.15. Örnek 6.11. ile ilgili tablo (1)

	T_1	T_2	T_3	T_4
a	40	7	12	9
b	12	7	30	19
c	20	8	13	25
d	8	5	20	12
e	10	6	7	13
f	13	9	15	18

Örnek 6.11.

	T_1	T_2	T_3	T_4
a	40	7	12	9
b	12	7	30	19
c	20	8	13	25
d	8	5	20	12
e	10	6	7	13
f	13	9	15	18

Bu işlerin yayılma süresini enküçükleyecek veya buna yakın iyi bir şekilde sıralanması için yapılan işlemler (CDS algoritmasının uygulanması) sonucu bulunan ara sonuçlar da Tablo 6.16., Tablo 6.17. ve Tablo 6.18.'de verilmiştir. Görüldüğü gibi bu problemde (4–1=) 3 çözüm grubu elde edilebilir.

Tablo 6.16.

	Küme-1 T_1	Küme-2 T_4
a	40	9
b	12	19
c	20	25
d	8	12
e	10	13
f	13	18

Tablo 6.17.

	Küme-1 T_1+T_2	Küme-2 T_3+T_4
a	47	21
b	19	49
c	28	38
d	13	32
e	16	20
f	22	33

Örnek 6.11.

	T_1	T_2	T_3	T_4
a	40	7	12	9
b	12	7	30	19
c	20	8	13	25
d	8	5	20	12
e	10	6	7	13
f	13	9	15	18

Tablo 6.18.

	Küme-1 $T_1+T_2+T_3$	Küme-2 $T_2+T_3+T_4$
a	59	28
b	49	56
c	41	46
d	33	37
e	23	26
f	37	42

Tablo 6.16. ve Tablo 6.17.: J1-A yönteminin uygulanması sonucu

(d-e-b-f-c-a)

Tablo 6.18.: J1-A yönteminin uygulanması sonucu **(e-d-f-c-b-a)**

Örnek 6.11.

Elde edilen iş sıralarının Gantt diyagramları çizilip yayılma süreleri hesaplandığında, şu sonuçlar elde edilir:

Sıralama Durumu	Yayılma Süresi
d-e-b-f-c-a	141
e-d-f-c-b-a	131

(**e-d-f-c-b-a**) iş sırasının M değeri daha az, çözüm bu iş sırası.

Dal-Sınır Algoritması ile (n×m) Sıralama Problemlerinin Çözümü

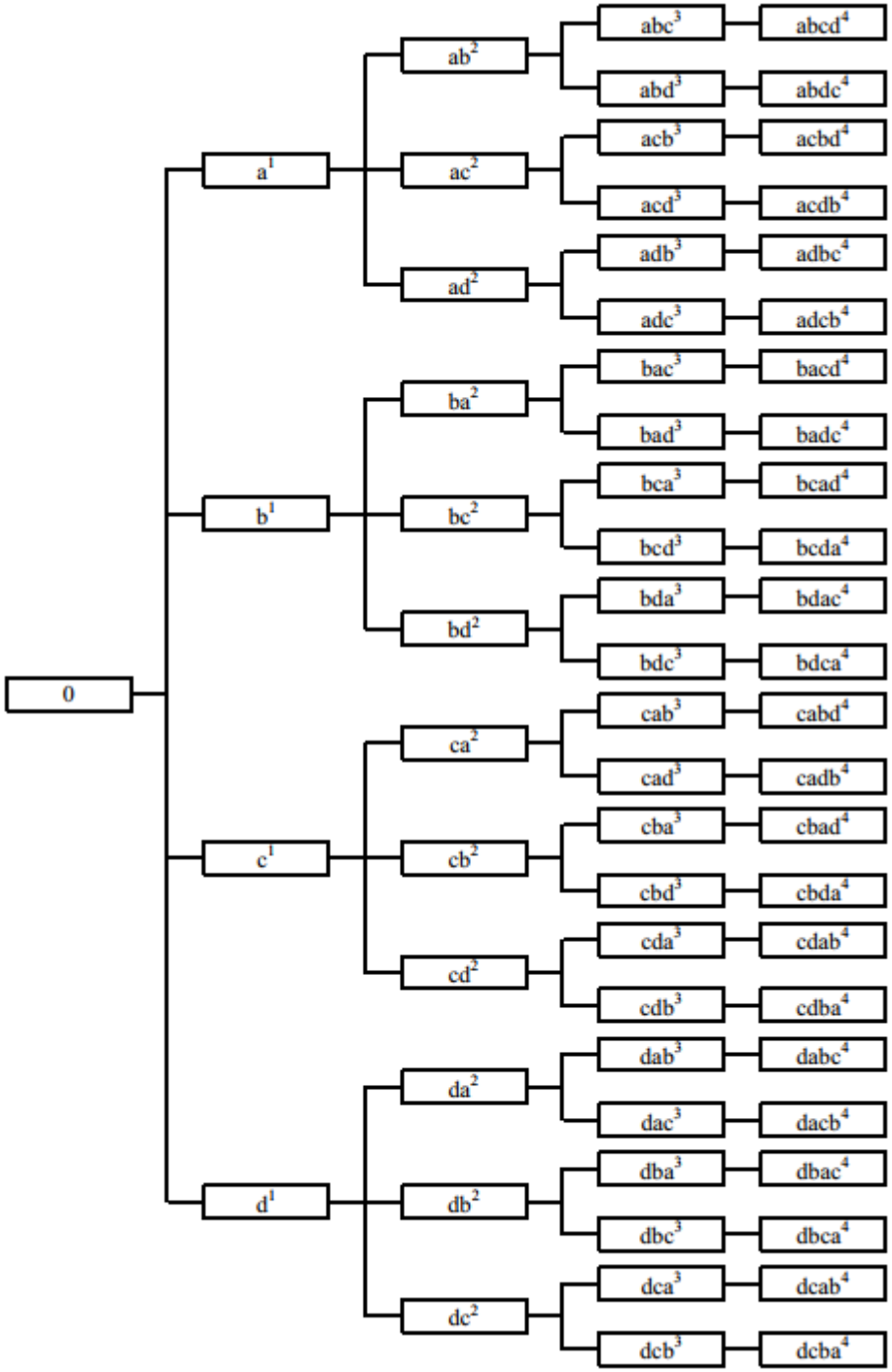
- Ignall ve Schrage'in geliştirdiği dal-sınır algoritması, özellikle küçük ölçekli (n×m) sıralama problemlerinin çözümünde yararlı olmaktadır [(n×3) tipi sıralama problemlerine de uygulanabilir].

- Gerek dal-sınır algoritması, gerekse tamsayılı programlama modelleri, optimum sonucu bulabilir. Ama optimum çözüm gerekli midir ? Bu da sorulması gereken bir sorudur. Çünkü optimum çözümü bulmak için uzun hesaplamalar yapmak gerekmektedir. Dolayısıyla uygulamada kullanışlı olup olmadığı tartışmalı bir konudur.

Dal-Sınır Algoritması ile (n×m) Sıralama Problemlerinin Çözümü

- Bu algoritmayı, tüm tezgâhlardaki işlem sırasının aynı olması durumunda uygulamak olanaklıdır. Problem (n!) tane farklı iş sırasından, yayılma süresi en az olanı seçmek problemine dönüşmektedir.

- Dal-sınır algoritmasının diğer uygulamalarında olduğu gibi, (n!) tane değişik sıralamadan bâzıları, yinelemenin ilk adımlarında elenerek, daha kısa sürede çözüme ulaşmak olanaklı olmaktadır.



Şekil 6.5. 4 işin n tezgâhtan sırasıyla geçmesi durumu için oluşturulan karar ağacı

Burada görüldüğü gibi (4!=) 24 farklı olası sıra vardır.

Dal-Sınır Algoritması ile (n×m) Sıralama Problemlerinin Çözümü

- Karar ağacı diyagramındaki her bir düğüm, o dala kadar gelen işlerin numaralarını içeren birkaç rakam ile tanımlanacaktır. Bu dallandırma işleminden sonra sıra ağaç diyagram üstündeki her bir düğüme veya bu düğümdeki ilgili iş sırasına, toplam işleme süresi için bir alt-sınır değeri koyma işlemine gelir.

Dal-Sınır Algoritması ile (n×m) Sıralama Problemlerinin Çözümü

- Problemde kaç tane tezgâh sözkonusu ise, o kadar alt-sınır tanımı yapılır; bu alt-sınırlardan en büyük olanı, sözkonusu düğüm için alt-sınır olarak alınır.

- Örnek olarak 4 tezgâh durumunu ele alalım:

Dal-Sınır Algoritması ile (n×m) Sıralama Problemlerinin Çözümü

X	Herhangi bir ön sıra [Örneğin 1-2-3-4 işleri için 1-3 ön sırası]
Y	X ön sırasında olmayan tüm geri kalan işler kümesi [Örnek için 2-4]
q_i	X ön sırasındaki en son işin i. tezgâhtaki bitiş zamanı (i=1, 2, 3, 4)
b_i	X ön sırası için i. tezgâh alt-sınır değeri (i=1, 2, 3, 4)

$$q_1 = \sum_{i \in X} P_{1i}$$

$$b_1 = q_1 + \sum_{i \in Y} P_{i1} + \underbrace{Enk}_{i \in Y} (P_{i2} + P_{i3} + P_{i4})$$

$$b_2 = q_2 + \sum_{i \in Y} P_{i2} + \underbrace{Enk}_{i \in Y} (P_{i3} + P_{i4})$$

$$b_3 = q_3 + \sum_{i \in Y} P_{i3} + \underbrace{Enk}_{i \in Y} \mathrm{P}_{i4}$$

$$b_4 = q_4 + \sum_{i \in Y} P_{i4}$$

Dal-Sınır Algoritması ile (n×m) Sıralama Problemlerinin Çözümü

b_i değerlerinden yararlanarak, bir ön sıraya ilişkin alt-sınır değeri şöyle hesaplanır:

$$\boldsymbol{B = Enb(b_1; b_2; b_3; b_4)}$$

Herhangi bir sıraya yerleştirilecek iş, o sıra için hesaplanan B değerlerinin en küçüğünden yararlanarak belirlenir ve o düğümden yola çıkılarak bir sonraki sırada yapılacak olan iş saptanmaya çalışılır.

Örnek 6.12.

4 iş (i), 4 tezgâhtan (T_1-T_2-T_3-T_4) tezgâh sırasıyla geçerek işlenmektedir.

Bu işlere ilişkin işlem süreleri, Tablo 6.21.'de verilmiştir.

Tablo 6.21. Örnek 6.12. ile ilgili tablo (1)

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

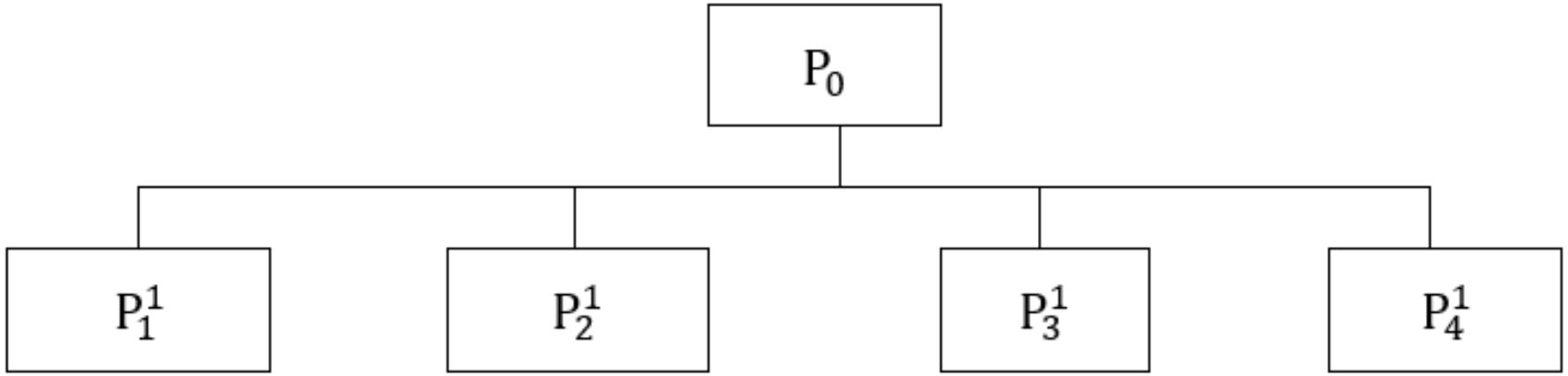
Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

Dal-sınır algoritması ile optimal iş sırasını bulmaya çalışalım.

Önce 1. kademe düğümler ve onların alt-sınır değerlerini hesaplayalım.

1. kademe düğümler, aşağıdaki şekildeki gibidir.



Örnek 6.12. ile ilgili 1. kademe düğümleri

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

Şimdi alt sınır değerlerinin nasıl hesaplandığına bakalım:

P_1^1 düğümü :

1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

b_1 = 5 + (4+9+6) + Enk [(7+3+5); (5+5+3); (8+4+1)] = 5 + 19 + 13 = 37
b_2 = (5+6) + (7+5+8) + Enk [(3+5); (5+3); (4+1)] = 11 + 20 + 5 = 36
b_3 = (5+6+4) + (3+5+4) + Enk (5; 3; 1) = 15 + 12 + 1 = 28
b_4 = (5+6+4+5) + (5+3+1) = 20 + 9 = 29
$B = Enb(b_1; b_2; b_3; b_4) = 37$

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

P_2^1 düğümü :

2	4	7	3	5
1	5	6	4	5
3	9	5	5	3
4	6	8	4	1

b_1 = 4 + (5+9+6) + Enk [(6+4+5); (5+5+3); (8+4+1)] = 4 + 20 + 13 = 37
b_2 = (4+7) + (6+5+8) + Enk [(4+5); (5+3); (4+1)] = 11 + 19 + 5 = 35
b_3 = (4+7+3) + (4+5+4) + Enk (5; 3; 1) = 14 + 13 + 1 = 28
b_4 = (4+7+3+5) + (5+3+1) = 19 + 9 = 28
$B = Enb(b_1; b_2; b_3; b_4) = 37$

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

P_3^1 düğümü :

3	9	5	5	3
1	5	6	4	5
2	4	7	3	5
4	6	8	4	1

b_1 = 9 + (5+4+6) + Enk [(6+4+5); (7+3+5); (8+4+1)] = 9 + 15 + 13 = 37
b_2 = (9+5) + (6+7+8) + Enk [(4+5); (3+5); (4+1)] = 14 + 21 + 5 = 40
b_3 = (9+5+5) + (4+3+4) + Enk (5; 5; 1) = 19 + 11 + 1 = 31
b_4 = (9+5+5+3) + (5+5+1) = 22 + 11 = 33
$B = Enb(b_1; b_2; b_3; b_4) = 40$

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

P_4^1 düğümü :

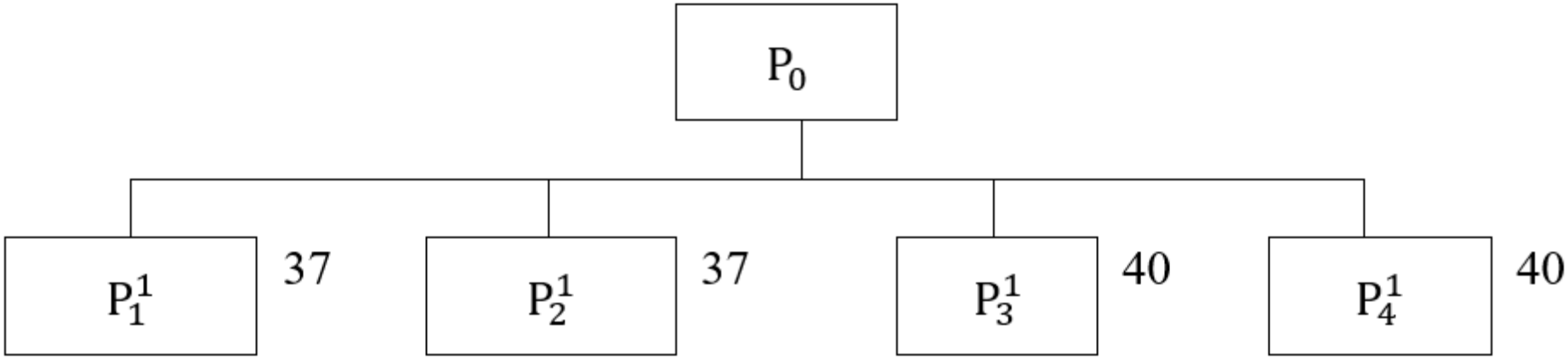
4	6	8	4	1
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3

b_1 = 6 + (5+4+9) + Enk [(6+4+5); (7+3+5); (5+5+3)] = 6 + 18 + 13 = 37
b_2 = (6+8) + (6+7+5) + Enk [(4+5); (3+5); (5+3)] = 14 + 18 + 8 = 40
b_3 = (6+8+4) + (4+3+5) + Enk (5; 5; 3) = 18 + 12 + 3 = 33
b_4 = (6+8+4+1) + (5+5+3) = 19 + 13 = 32
$B = Enb(b_1; b_2; b_3; b_4) = 40$

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

Şekil 6.6.'da; 1. kademe düğümler ve onların alt-sınır değerleri toplu olarak gösterilmektedir.

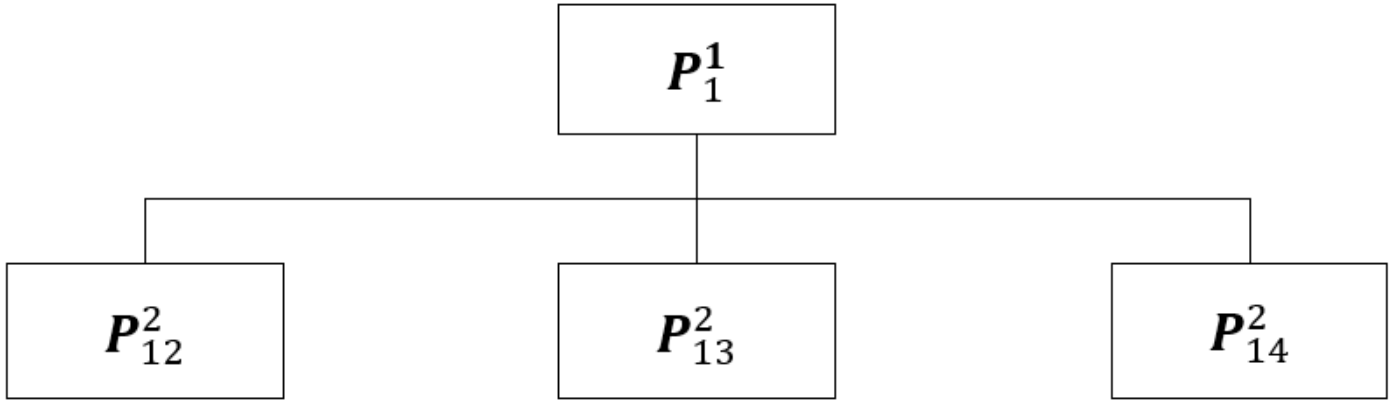


Şekil 6.6. Örnek 6.12. ile ilgili düğümler ve alt sınır değerleri (1)

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

- ✓ Görüldüğü gibi en düşük alt-sınır değeri $\boldsymbol{P_1^1}$ ve $\boldsymbol{P_2^1}$ düğümlerine ilişkin. Bunlardan birini, örneğin $\boldsymbol{P_1^1}$ düğümünü alalım.
- ✓ 2. kademe düğümleri oluşturalım ve bu düğümlere ilişkin alt-sınır değerlerini hesaplayalım.
- ✓ $\boldsymbol{P_1^1}$ düğümü ve buna bağlı 2. kademe düğümler, aşağıdaki şekildeki gibidir.



Örnek 6.12. ile ilgili 2. kademe düğümleri

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

Şimdi alt sınır değerlerinin nasıl hesaplandığına bakalım:

P_{12}^{2} düğümü :

1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

b_1 = (5+4) + (9+6) + Enk [(5+5+3); (8+4+1)] = 9 + 15 + 13 = 37

b_2 = Enb [(11+7); (9+7)] + (5+8) + Enk [(5+3); (4+1)] = 18 + 13 + 5 = 36

b_3 = Enb [(15+3); (18+3)] + (5+4) + Enk (3; 1) = 21 + 9 + 1 = 31

b_4 = Enb [(20+5); (21+5)] + (3+1) = 26 + 4 = 30

$B = Enb(b_1; b_2; b_3; b_4) = 37$

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

P_{13}^{2} düğümü :

1	5	6	4	5
3	9	5	5	3
2	4	7	3	5
4	6	8	4	1

b_1 = (5+9) + (4+6) + Enk [(7+3+5); (8+4+1)] = 14 + 10 + 13 = 37
b_2 = Enb [(11+5); (14+5)] + (7+8) + Enk [(3+5); (4+1)] = 19 + 15 + 5 = 39
b_3 = Enb [(15+5); (19+5)] + (3+4) + Enk (5; 1) = 24 + 7 + 1 = 32
b_4 = Enb [(20+3); (24+3)] + (5+1) = 27 + 6 = 33
$B = Enb(b_1; b_2; b_3; b_4) = 39$

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

P_{14}^{2} düğümü :

1	5	6	4	5
4	6	8	4	1
2	4	7	3	5
3	9	5	5	3

b_1 = (5+6) + (4+9) + Enk [(7+3+5); (5+5+3)] = 11 + 13 + 13 = 37

b_2 = Enb [(11+8); (11+8)] + (7+5) + Enk [(3+5); (5+3)] = 19 + 12 + 8 = 39

b_3 = Enb [(15+4); (19+4)] + (3+5) + Enk (5; 3) = 23 + 8 + 3 = 34

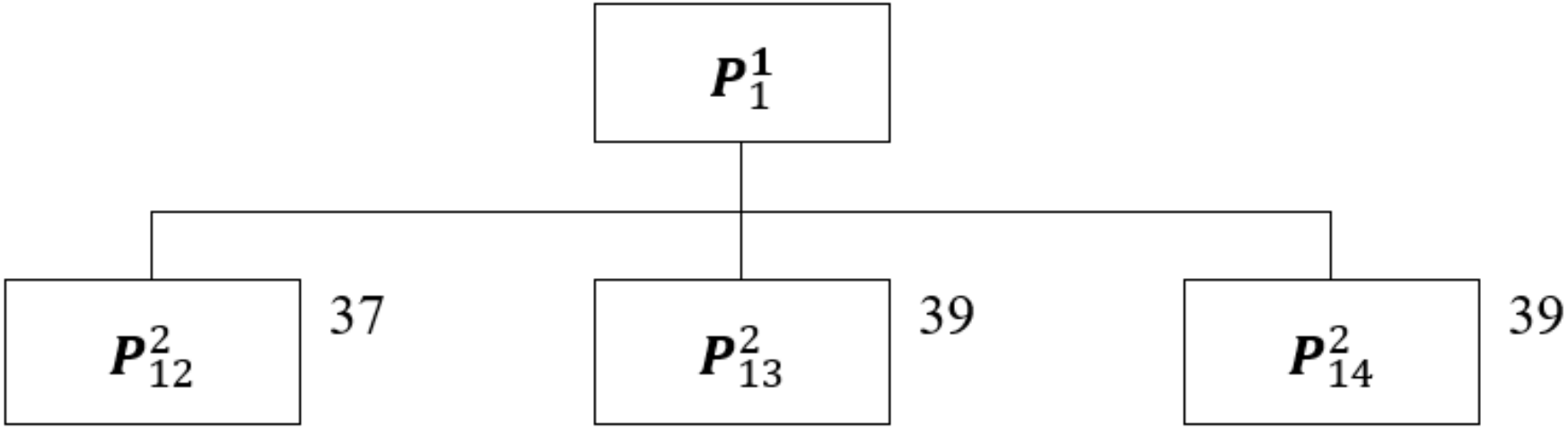
b_4 = Enb [(20+1); (23+1)] + (5+3) = 24 + 8 = 32

$B = Enb(b_1; b_2; b_3; b_4) = 39$

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

Şekil 6.7.'de $\boldsymbol{P_1^1}$ düğümü ve buna bağlı 2. kademe düğümler ile onların alt-sınır değerleri toplu olarak gösterilmektedir.

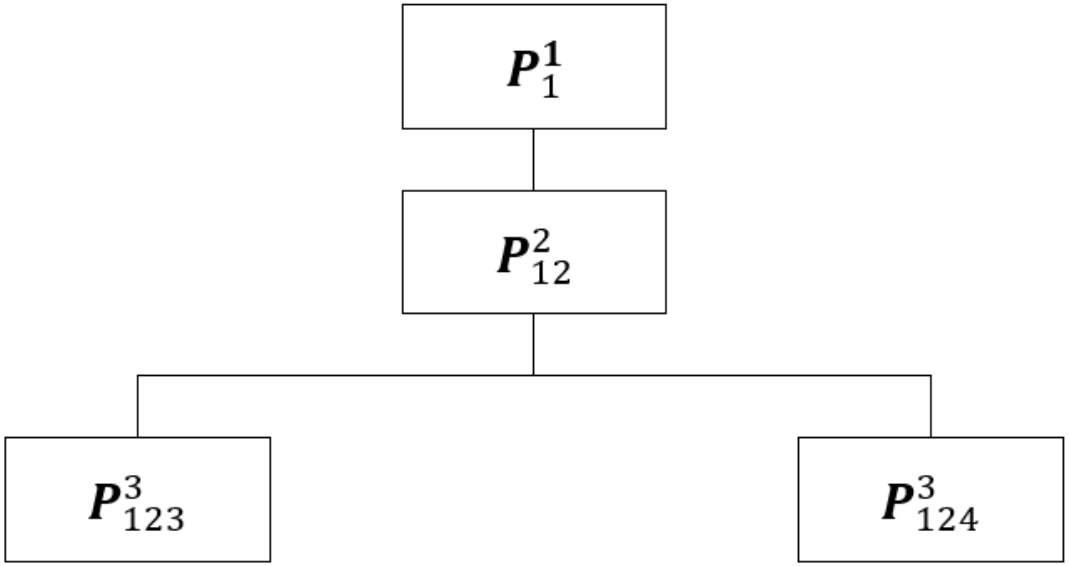


Şekil 6.7. Örnek 6.12. ile ilgili düğümler ve alt sınır değerleri (2)

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

- ✓ Görüldüğü gibi en düşük alt-sınır değeri $\boldsymbol{P_{12}^{2}}$ düğümüne ilişkin.
- ✓ Bu düğüm üzerinden ilerleyerek 3. kademe düğümleri oluşturalım ve bu düğümlere ilişkin alt-sınır değerlerini hesaplayalım.
- ✓ $\boldsymbol{P_{12}^{2}}$ düğümü ve buna bağlı 3. kademe düğümler, aşağıdaki şekildeki gibidir.



Örnek 6.12. ile ilgili 3. kademe düğümleri

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

Şimdi de bu alt sınır değerlerinin nasıl hesaplandığına bakalım:

P^3_{123} düğümü :

1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

b_1 = (5+4+9) + 6 + Enk (8+4+1) = 18 + 6 + 13 = 37
b_2 = Enb [(18+5); (18+5)] + 8 + Enk (4+1) = 23 + 8 + 5 = 36
b_3 = Enb [(21+5); (23+5)] + 4 + Enk (1) = 28 + 4 + 1 = 33
b_4 = Enb [(28+3); (26+3)] + 1 = 31 + 1 = 32
$B = Enb(b_1; b_2; b_3; b_4) = 37$

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

P^3_{124} düğümü :

1	5	6	4	5
2	4	7	3	5
4	6	8	4	1
3	9	5	5	3

b_1 = (5+4+6) + 9 + Enk (5+5+3) = 15 + 9 + 13 = 37

b_2 = Enb [(15+8); (18+8)] + 5 + Enk (5+3) = 26 + 5 + 8 = 39

b_3 = Enb [(26+4); (21+4)] + 5 + Enk (3) = 30 + 5 + 3 = 38

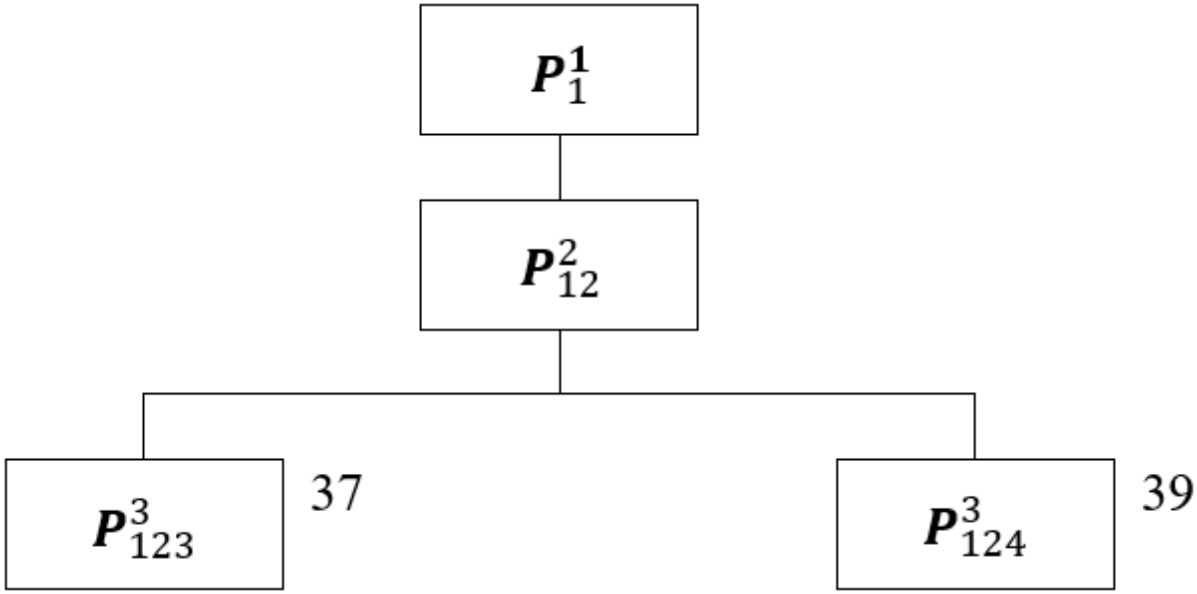
b_4 = Enb [(30+1); (26+1)] + 3 = 31 + 3 = 34

$B = Enb(b_1; b_2; b_3; b_4) = 39$

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

Şekil 6.8.'de $\boldsymbol{P_{12}^{2}}$ düğümü ve buna bağlı 3. kademe düğümler ile onların alt-sınır değerleri toplu olarak gösterilmektedir.



Şekil 6.8. Örnek 6.12. ile ilgili düğümler ve alt sınır değerleri (3)

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

- ✓ Görüldüğü gibi en düşük alt-sınır değeri $\boldsymbol{P_{123}^{3}}$ düğümüne ilişkin. Bu düğümün sonu 4 numaralı iş olup buradan çıkan sonuç (1-2-3-4) iş sırasının optimum iş sırası olduğudur.
- ✓ Dal-Sınır algoritmasının yapısı gereği, ara aşamalarda bulunan en alt sınır değerleri, daha önceki aşamalarda bulunan ve sürdürülmeyen düğümlerin en alt sınır değerinden daha büyük çıksaydı, önceki aşamadaki ilgili düğüm(ler)den işlemi sürdürmek gerekecekti.

Örnek 6.12.

i	T_1	T_2	T_3	T_4
1	5	6	4	5
2	4	7	3	5
3	9	5	5	3
4	6	8	4	1

- ✓ Bu örnek soru için örneğin 2. veya 3. kademe sonunda 37'den daha büyük bir alt sınır değeri elde edilmiş olsaydı, işlemleri P_2^1 düğümünden sürdürmek sözkonusu olacaktı. Burada bulunan nihaî en alt sınır değeri, o iş sırasının yayılma süresidir.
- ✓ Ayrıca bu sorunun verileri için CDS algoritmasının uygulanması sonucunda elde edilen (2-1-3-4) iş sırasının yayılma süresinin de, (1-2-3-4) sırasınınki ile aynı ve 37 olduğu görülmüştür.

(n×m) Sıralama Problemlerinde Yayılma Süresinin ROY Algoritması ile Hesabı

Örnek 6.13.

Tablo 6.22.'de verilen işler, T_1-T_2-T_3-T_4 tezgâh sırası ile yapılmaktadır.

CDS algoritması uygulandığında elde edilen en uygun sırası (c-d-f-b-e-a) ve yayılma süresi ise 487 olmaktadır.

Örnek 6.13.

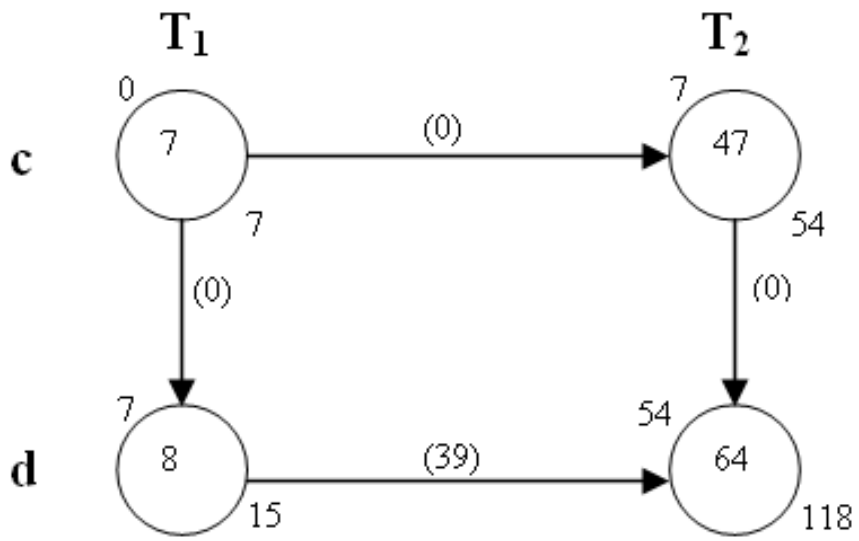
Tablo 6.22. Örnek 6.13. ile ilgili tablo

	T_1	T_2	T_3	T_4
c	7	47	20	98
d	8	64	12	84
f	1	80	66	78
b	89	99	95	77
e	61	19	65	14
a	50	43	15	4

Örnek 6.13.

	T_1	T_2	T_3	T_4
c	7	47	20	98
d	8	64	12	84
f	1	80	66	78
b	89	99	95	77
e	61	19	65	14
a	50	43	15	4

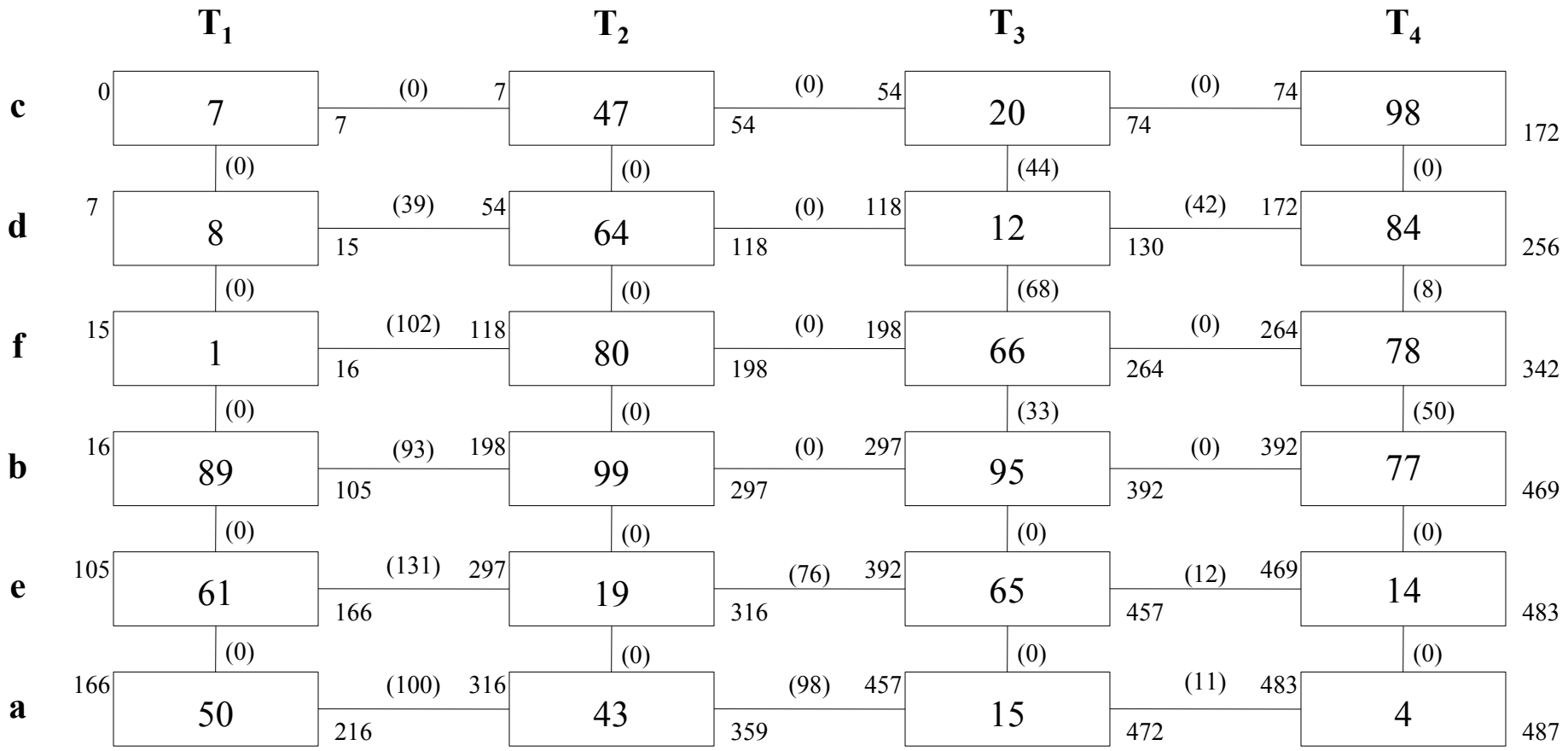
c ve d işleri ile T_1 ve T_2 tezgâhlarının bulunduğu (2×2)'lik matris bölgesini ele alalım. Bu bölgedeki süreler, matrisin düğümlerini oluşturacaktır. Bu bölge için EB (Erken Başlama) ve ES (Erken Sona Erme) değerleri ile tezgâh ve iş bekleme sürelerini hesaplarsak Şekil 7.16.'daki durumu elde ederiz.



Şekil 6.9. Örnek 6.13. ile ilgili ara çözüm

Örnek 6.13.

	T_1	T_2	T_3	T_4
c	7	47	20	98
d	8	64	12	84
f	1	80	66	78
b	89	99	95	77
e	61	19	65	14
a	50	43	15	4



Şekil 6.10. Örnek 6.13. ile ilgili genel çözüm

Örnek 6.13.

Görüldüğü gibi işler en erken 487 ZB'nde bitirilmektedir.

İşler için boş bekleme süreleri şu şekilde belirlenir:

c : 0 + 0 + 0 = 0

d : 39 + 0 + 42 = 81

f : 102 + 0 + 0 = 102

b : 93 + 0 + 0 = 93

e : 131 + 76 + 12 = 219

a : 100 + 98 + 11 = 209

Örnek 6.13.

Tezgâhlar için boş bekleme süreleri ise şöyle belirlenir:

1 : 0 + 0 + 0 + 0 + 0 = 0

2 : 0 + 0 + 0 + 0 + 0 = 0

3 : 44 + 68 + 33 + 0 + 0 = 145

4 : 0 + 8 + 50 + 0 + 0 = 58

Toplam iş bekleme süresi

0 + 81 + 102 + 93 + 219 + 209 = 704 ZB

olup toplam tezgâh boş bekleme süresi ise

0 + 0 + 145 + 58 = 203 ZB

olarak bulunur.

Bâzı Klasik Algoritmalar
1) Palmer Algoritması
Örnek Üzerinde Tanıtım

i	T_1	T_2	T_3
i_1	1	8	4
i_2	2	4	5
i_3	6	2	8
i_4	3	9	2

Makina Sayısı = 3 **(Katsayılar: -2,0,2)**

$s_1 = -(3-1)\, t_{11} - (3-3)\, t_{12} + (3-1)\, t_{13}$

$s_2 = -(3-1)\, t_{21} - (3-3)\, t_{22} + (3-1)\, t_{23}$

$s_3 = -(3-1)\, t_{31} - (3-3)\, t_{32} + (3-1)\, t_{33}$

$s_4 = -(3-1)\, t_{41} - (3-3)\, t_{42} + (3-1)\, t_{43}$

$\mathbf{s_1 = -2\times1 - 0\times8 + 2\times4 = 6}$

$\mathbf{s_2 = -2\times2 - 0\times4 + 2\times5 = 6}$

$\mathbf{s_3 = -2\times6 - 0\times2 + 2\times8 = 4}$

$\mathbf{s_4 = -2\times3 - 0\times9 + 2\times2 = -2}$

Uygun Sıra: s_i'lere göre büyükten küçüğe olan sıra:

1-2-3-4 VEYA **2-1-3-4**

Yayılma Süresi = 28 (her iki durum için de)

OPTİMUM OLMA GARANTİSİ YOK

Bâzı Klasik Algoritmalar
2) Gupta Algoritması
Örnek Üzerinde Tanıtım

i	T_1	T_2	T_3
i_1	1	8	4
i_2	2	4	5
i_3	6	2	8
i_4	3	9	2

$$s_i = \frac{e_i}{\min\limits_{1 \le k \le m-1} \{t_{ik} + t_{i,k+1}\}}$$

$$e_i = \begin{cases} 1 & \text{if } t_{i1} < t_{im} \\ -1 & \text{if } t_{i1} \ge t_{im} \end{cases}$$

Enk {1+8, 8+4} = 9, e_1=1, s_1=1/9

Enk {2+4, 4+5} = 6, e_2=1, s_2=1/6

Enk {6+2, 2+8} = 8, e_3=1, s_3=1/8

Enk {3+9, 9+2} = 11, e_4=-1, s_4=-1/11

Uygun Sıra: s_i'lere göre büyükten küçüğe olan sıra:

2-3-1-4

Yayılma Süresi = 29

OPTİMUM OLMA GARANTİSİ YOK

Bâzı Klasik Algoritmalar
3) Dannenbring Algoritması
Örnek Üzerinde Tanıtım

i	T_1	T_2	T_3
i_1	1	8	4
i_2	2	4	5
i_3	6	2	8
i_4	3	9	2

Makina Sayısı = m

$$a_i = m \times t_{i1} + (m-1) \times t_{i2} + \ldots + 2 \times t_{i(m-1)} + 1 \times t_{im}$$

$$b_i = 1 \times t_{i1} + 2 \times t_{i2} + \ldots + (m-1) \times t_{i(m-1)} + m \times t_{im}$$

$$a_1 = 3\times1+2\times8+1\times4 = 23$$

$$a_2 = 3\times2+2\times4+1\times5 = 19$$

$$a_3 = 3\times6+2\times2+1\times8 = 30$$

$$a_4 = 3\times3+2\times9+1\times2 = 29$$

$$b_1 = 1\times1+2\times8+3\times4 = 29$$

$$b_2 = 1\times2+2\times4+3\times5 = 25$$

$$b_3 = 1\times6+2\times2+3\times8 = 34$$

$$b_4 = 1\times3+2\times9+3\times2 = 27$$

i	a_i	b_i
i_1	23^*	29
i_2	19^*	25
i_3	30^*	34
i_4	29	27^*

Uygun Sıra:

J1A veya J1B'ye göre

2-1-3-4

Yayılma Süresi = 28

Bâzı Klasik Algoritmalar
4) NEH (Nawaz-Enscore-Ham) Algoritması
Örnek Üzerinde Tanıtım

i	T_1	T_2	T_3	T_4
i_1	25	45	52	40
i_2	7	41	22	66
i_3	41	55	33	21
i_4	74	12	24	48
i_5	7	15	72	52
i_6	12	14	22	32

1) Her işin işlem süreleri toplamı hesaplanır:

i	$\sum t_{ij}$
i_1	162
i_2	136
i_3	150
i_4	158
i_5	146
i_6	80

2) İşler $\sum t_{ij}$ değerlerine göre büyükten küçüğe sıralanır:

i	$\sum t_{ij}$
i_1	162
i_4	158
i_3	150
i_5	146
i_2	136
i_6	80

Bâzı Klasik Algoritmalar
4) NEH (Nawaz-Enscore-Ham) Algoritması
Örnek Üzerinde Tanıtım

i	T_1	T_2	T_3	T_4
i_1	25	45	52	40
i_4	74	12	24	48
i_3	41	55	33	21
i_5	7	15	72	52
i_2	7	41	22	66
i_6	12	14	22	32

3) İlk iki sıradaki iş alınır ve kendi aralarında yer değiştirerek farklı sıralar üretilir. Her bir sıra için yayılma süresi hesaplanır. Daha küçük süreli olan yeğlenir:

1-4: M=210

4-1: M=236

1-4 sırası uygun

Bâzı Klasik Algoritmalar
4) NEH (Nawaz-Enscore-Ham) Algoritması
Örnek Üzerinde Tanıtım

i	T_1	T_2	T_3	T_4
i_1	25	45	52	40
i_4	74	12	24	48
i_3	41	55	33	21
i_5	7	15	72	52
i_2	7	41	22	66
i_6	12	14	22	32

4) Adım-3'de elde edilen sıraya, 2. adımda elde edilen sıralamada 3. sırada bulunan iş katılır. Bu iş, adım-3'de elde edilen sıraya çeşitli yerlerden eklenerek yeni sıralar oluşturulur:

Eklenecek iş: 3

3-1-4: M=281

1-3-4: M=231

1-4-3: M=249

1-3-4 sırası uygun

Bâzı Klasik Algoritmalar
4) NEH (Nawaz-Enscore-Ham) Algoritması
Örnek Üzerinde Tanıtım

i	T_1	T_2	T_3	T_4
i_1	25	45	52	40
i_3	41	55	33	21
i_4	74	12	24	48
i_5	7	15	72	52
i_2	7	41	22	66
i_6	12	14	22	32

5) Yeni, eklenecek iş: 5

5-1-3-4: M=255

1-5-3-4: M=315

1-3-5-4: M=330

1-3-4-5: M=306

5-1-3-4 sırası uygun

Bâzı Klasik Algoritmalar
4) NEH (Nawaz-Enscore-Ham) Algoritması
Örnek Üzerinde Tanıtım

i	T_1	T_2	T_3	T_4
i_5	7	15	72	52
i_1	25	45	52	40
i_3	41	55	33	21
i_4	74	12	24	48
i_2	7	41	22	66
i_6	12	14	22	32

6) Yeni, eklenecek iş: 2

2-5-1-3-4: M=304

5-2-1-3-4: M=320

5-1-2-3-4: M=321

5-1-3-2-4: M=321

5-1-3-4-2: M=321

2-5-1-3-4 sırası uygun

Bâzı Klasik Algoritmalar
4) NEH (Nawaz-Enscore-Ham) Algoritması
Örnek Üzerinde Tanıtım

i	T_1	T_2	T_3	T_4
i_2	7	41	22	66
i_5	7	15	72	52
i_1	25	45	52	40
i_3	41	55	33	21
i_4	74	12	24	48
i_6	12	14	22	32

7) Yeni ve en son eklenecek iş: 6

6-2-5-1-3-4: M=**322**

2-6-5-1-3-4: M=332

2-5-6-1-3-4: M=336

2-5-1-6-3-4: M=336

2-5-1-3-6-4: M=336

2-5-1-3-4-6: M=336

SON SIRA: 6-2-5-1-3-4 sırası uygun